

Introduction

What is LLD(Low level Design) ?

LLD, or Low-Level Design, is a phase in the software development process where detailed system components and their interactions are specified, along with their implementation. It involves converting the high-level design into a more detailed blueprint, addressing specific algorithms, data structures, and interfaces. LLD serves as a guide for developers during coding, which ensures the accurate and efficient implementation of the system's functionality.

What are the types of LLD interviews ?

There are mainly 3 types of interviews in this field, according to my knowledge.

1. Low Level Design Interview (make class Diagram, use case diagram)
2. Machine Coding Round (write the complete working code)
3. Database Design Round (just make all the database tables and their)

1. Low-Level Design (LLD) Interview

- **Objective:** Evaluate your ability to **design object-oriented systems** using **class diagrams**, **use case diagrams**, **design patterns** and **OOP principles**.
- **Focus Areas:**
 - Identifying **entities** and their relationships
 - Designing **class structures** and defining **attributes/methods**
 - Following **SOLID principles** and **design patterns**
 - Handling **scalability** and **extensibility**
- **Common Questions:**
 - **Design a Parking Lot system**
 - **Design a Library Management system**
 - **Design an ATM system**
 - **Design a Hotel Booking system**

2. Machine Coding Round

- **Objective:** Assess your ability to **write a fully functional, modular, and maintainable** system in a limited timeframe. (LLD Vaala pura, saath mei pura ka pura running code)
- **Focus Areas:**
 - Writing **clean and modular code**
 - Implementing **business logic correctly**
 - Handling **edge cases and errors**
 - Ensuring **OOP best practices**
 - Writing **test cases**
- **Common Questions:**
 - **Develop a Snake and Ladder game**
 - **Implement a Cache System (LRU Cache)**
 - **Build a Ride-Sharing System (like Uber)**
 - **Create a Multi-Level Tic-Tac-Toe Game**

3. Database Design Round

- **Objective:** Evaluate your ability to **design efficient, normalized database schemas** for real-world applications.
- **Focus Areas:**
 - Identifying **tables and relationships**
 - **Choosing the right data types and constraints**
 - **Normalization (1NF, 2NF, 3NF, BCNF)**
 - Handling **Indexes, Partitioning, and Scaling**
- **Common Questions:**
 - **Design a database for an E-commerce platform**
 - **Design a schema for a Flight Booking System**
 - **Design a database for a Social Media Platform (like Twitter/Facebook)**
 - **Create a schema for a Food Delivery App**

Topics To Study

1. OOPS
2. Solid Principles
3. Object Oriented Design and Analysis
4. Design Patterns
5. LLD Interview Questions

OOPS

1. What is OOP?

- Difference between OOP vs Procedural Programming
- Advantages of OOP (Modularity, Code Reusability, Scalability)

2. Basic Terminologies

1. Class & Object
 - a. Definition, Example, Memory Allocation
2. Methods (Member Functions)
3. Access Modifiers
 - a. public, private, protected
4. Constructors & Destructors
5. Static Members and Functions
6. Friend Functions & Friend Classes
7. Shallow Copy vs Deep Copy
 - a. Understanding Copy Constructors
 - b. Using = delete to prevent copying
8. Smart Pointers
9. unique_ptr, shared_ptr, weak_ptr
10. Exception Handling in OOP
11. try-catch blocks
12. Custom Exception classes

3. Encapsulation (Data Hiding & Data Protection)

Encapsulation is **binding data and methods together** to protect data from unauthorized access.

a. How to implement Encapsulation in C++?

- Using **private** members
- Providing **getter** and **setter** functions

```
class BankAccount {  
private:  
    double balance; // Private data  
  
public:  
    void setBalance(double amount) { balance = amount; }  
    double getBalance() { return balance; }  
};
```

b. Access Modifiers in Detail

- **private**: Accessible only inside the class
- **protected**: Accessible in derived classes
- **public**: Accessible everywhere

c. Benefits of Encapsulation

- Improves security (Data hiding)
- Code maintainability (Data controlled via methods)

4. Inheritance (Code Reusability & Hierarchy)

Inheritance allows a **class to acquire properties and behavior from another class**.

a. Types of Inheritance

- Single Inheritance – One parent, one child
- Multiple Inheritance – A child inherits from multiple parents
- Multilevel Inheritance – Parent → Child → Grandchild
- Hierarchical Inheritance – One parent, multiple children
- Hybrid Inheritance – Combination of two or more types

```
class Parent {  
public:  
    void show() { cout << "Parent class function"; }  
};
```

```
class Child : public Parent { // Inheriting Parent
};
```

b. Access Control in Inheritance

Parent Modifier	public Inheritance	protected Inheritance	private Inheritance
public	public	protected	private
protected	protected	protected	private
private	Not Inherited	Not Inherited	Not Inherited

c. Virtual Base Class (Avoiding Diamond Problem in Multiple Inheritance)

d. Constructor and Destructor in Inheritance

- Order of Execution in Multilevel Inheritance

e. Overriding Parent Methods

- `super` keyword equivalent (`Parent::method()` in C++)

f. Composition vs Inheritance (When to use what?)

5. Polymorphism (Same Interface, Different Behavior)

Polymorphism means **one name, multiple forms**.

a. Compile-Time Polymorphism (Static Binding)

Function Overloading

- Defining multiple functions with **same name but different parameters**

```
cpp
CopyEdit
class Math {
```

```

public:
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
};

```

4.2 Operator Overloading

- Overloading operators like +, -, *, etc.

```

cpp
CopyEdit
class Complex {
public:
    int real, imag;
    Complex operator + (Complex obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }
};

```

4.3 Constructor Overloading

(B) Run-Time Polymorphism (Dynamic Binding)

4.4 Function Overriding (Method Overriding in Derived Class)

- **Virtual Functions**
- **V-Table & V-Ptr (Virtual Table Mechanism in C++)**
- **Pure Virtual Functions & Abstract Classes**

Example:

```

cpp
CopyEdit
class Animal {
public:

```

```

        virtual void makeSound() { cout << "Animal Sound"; } //
Virtual function
};

class Dog : public Animal {
public:
    void makeSound() override { cout << "Bark"; } // Overriding
};

```

4.5 Dynamic Memory Allocation with Polymorphism

```

cpp
CopyEdit
Animal* obj = new Dog();
obj->makeSound(); // Calls Dog's makeSound() because of dynamic
binding

```

4.6 Function Overloading vs Function Overriding

5. Abstraction (Hiding Implementation Details)

Abstraction means **hiding unnecessary details and exposing only essential features**.

5.1 How to achieve Abstraction in C++?

- Using Abstract Classes (With Pure Virtual Functions)
- Using Interfaces (Implemented via Abstract Classes in C++)

Example of Abstract Class:

```

cpp
CopyEdit
class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};

class Circle : public Shape {
public:

```

```
void draw() override { cout << "Drawing Circle"; }
};
```

5.2 Difference Between Abstraction and Encapsulation

Feature	Encapsulation	Abstraction
Purpose	Data hiding	Implementation hiding
Implementation	Using private members & getters/setters	Using abstract classes & interfaces
Example	Bank account balance hidden from direct access	Only essential functions exposed

Solid Principles

Gfg Link - [SOLID Principles in Programming: Understand With Real Life Examples - GeeksforGeeks](#)

- Single Responsibility Principle (SRP)**
 - A class should have only **one reason to change**.
 - Each class should only focus on a **single functionality**.
 - Example: A `User` class should not handle both user data and sending emails. Instead, an `EmailService` should handle emails separately.
- Open-Closed Principle (OCP)**
 - A class should be **open for extension but closed for modification**.
 - Instead of modifying existing code, we should extend it using **inheritance or interfaces**.
 - Example: Instead of modifying a `Payment` class to support new payment methods, create new subclasses like `CreditCardPayment` and `UPIPayment`.
- Liskov Substitution Principle (LSP)**
 - Subclasses should be replaceable** by their parent class without affecting the program.
 - A derived class should **not break the behavior** of the base class.

- c. Example: If **Bird** has a **fly()** method, then **Penguin** (a subclass of **Bird**) should not inherit it since penguins can't fly.
- 4. **Interface Segregation Principle (ISP)**
 - a. **No class should be forced** to implement interfaces it doesn't use.
 - b. Large interfaces should be **split into smaller, specific interfaces**.
 - c. Example: Instead of a single **Worker** interface with **work()** and **eat()**, split it into **Workable** (for work-related methods) and **Eatable** (for eating-related methods).
- 5. **Dependency Inversion Principle (DIP)**
 - a. **High-level modules should not depend on low-level modules**. Both should depend on **abstractions**.
 - b. Instead of directly creating objects, use **dependency injection** to inject dependencies.
 - c. Example: A **DatabaseService** should depend on an abstract **IDatabase** interface instead of a specific **MySQLDatabase**. This allows easy switching to other databases like **PostgreSQLDatabase**.

Object Oriented Design and Analysis

GFG Link - [Object-Oriented Analysis and Design\(OOAD\) - GeeksforGeeks](#)

1. Object-Oriented Analysis (OOA) – "What to Build?"

In this phase, we **analyze the problem domain** and **identify key objects, their attributes, and behaviors** based on real-world requirements.

Key Steps in OOA:

1. **Gather Requirements** – Understand user needs, system functionality, constraints.
2. **Identify Objects & Classes** – Extract real-world entities (e.g., "User", "Order", "Product").
3. **Define Relationships** – Establish how objects interact (e.g., aggregation, association, composition).
4. **Use Case Diagrams** – Model user interactions with the system.

💡 **Example (E-commerce System Analysis)**

1. Objects: `User`, `Product`, `Order`
2. Attributes: `User` (`name`, `email`), `Product` (`name`, `price`), `Order` (`date`, `status`)
3. Relationships: `User` places `Order`, `Order` contains `Product`

2. Object-Oriented Design (OOD) – "How to Build?"

In this phase, we **convert the analysis into a software architecture** using OOP principles.

Key Steps in OOD:

1. **Define Class Diagrams** – Convert objects into classes with attributes & methods.
2. **Apply OOP Principles** – Encapsulation, Inheritance, Polymorphism, Abstraction.
3. **Design Patterns** – Apply patterns like Factory, Singleton, Observer for better structure.
4. **Component & Deployment Models** – Define how different modules interact.

Unified Modeling Language (UML) Diagrams – Overview

UML (Unified Modeling Language) is a standardized way to visually represent a system's design. It helps in understanding, designing, and documenting software architecture.

Two Main Categories of UML Diagrams

1. **Structural Diagrams** – Describe the static structure of a system (what it contains).
2. **Behavioral Diagrams** – Describe the dynamic behavior of a system (how it interacts).

Structural UML Diagrams (Static Structure)

1. **Class Diagram** - <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/>
2. Object Diagram
3. Component Diagram
4. Deployment Diagram
5. Package Diagram
6. Composite Structure Diagram

Behavioral UML Diagrams (Dynamic Behavior)

1. **Use Case Diagram** - <https://www.geeksforgeeks.org/use-case-diagram/>
2. **Sequence Diagram** - <https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/>

3. Activity Diagram -

<https://www.geeksforgeeks.org/unified-modeling-language-uml-activity-diagrams/>

4. State Machine Diagram
5. Communication Diagram
6. Interaction Overview Diagram
7. Timing Diagram

Design Patterns

Software design patterns are important tools developers provide, providing proven solutions to common problems encountered during software development. This article will act as a tutorial to help you understand the concept of design patterns. Developers can create more robust, maintainable, and scalable software systems by understanding and applying these patterns.

GFG Link - [Software Design Patterns Tutorial - GeeksforGeeks](#)

Types of Design Pattern

1. Creational - [Creational Pattern in C++ - GeeksforGeeks](#) , [Creational Design Patterns - GeeksforGeeks](#)
2. Structural - <https://www.geeksforgeeks.org/structural-design-patterns/>
3. Behavioral - <https://www.geeksforgeeks.org/behavioral-design-patterns/>

Creational Design Pattern

1. Factory Method -

- a. <https://www.geeksforgeeks.org/factory-method-for-designing-pattern/?ref=lbp> (java)
- b. <https://www.geeksforgeeks.org/factory-method-pattern-c-design-patterns/> (C++)

2. Abstract Factory Method -

- a. <https://www.geeksforgeeks.org/abstract-factory-pattern/?ref=lbp> (java)
- b. <https://www.geeksforgeeks.org/abstract-factory-pattern-c-design-patterns/> (C++)

3. Singleton Pattern -

- a. <https://www.geeksforgeeks.org singleton-design-pattern/?ref=lbp> (java)
- b. <https://www.geeksforgeeks.org singleton-pattern-c-design-patterns/?ref=lbp> (C++)

4. Builder Pattern -

- a. <https://www.geeksforgeeks.org/builder-design-pattern/?ref=lbp> (java)

- b. <https://www.geeksforgeeks.org/builder-pattern-c-design-patterns/?ref=lbp> (C++)
- 5. **Prototype Pattern -**
 - a. <https://www.geeksforgeeks.org/prototype-design-pattern/?ref=lbp> (Java)
 - b. <https://www.geeksforgeeks.org/prototype-pattern-c-design-patterns/?ref=lbp> (C++)

Structural Design Pattern

- 1. **Adapter Pattern -**
 - a. <https://www.geeksforgeeks.org/adapter-design-pattern-in-java/> (java)
 - b. <https://www.geeksforgeeks.org/adapter-pattern/> (C++)
- 2. **Bridge Pattern -**
 - a. <https://www.geeksforgeeks.org/bridge-design-pattern/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/bridge-method-c-design-patterns/?ref=lbp> (C++)
- 3. **Composite Pattern -**
 - a. <https://www.geeksforgeeks.org/composite-method-software-design-pattern/?ref=lbp> (C++)
 - b. <https://www.geeksforgeeks.org/composite-pattern-cpp/?ref=lbp> (C++)
 - c. <https://www.geeksforgeeks.org/composite-design-pattern-in-java/> (Java)
- 4. **Decorator Pattern -**
 - a. <https://www.geeksforgeeks.org/decorator-design-pattern-in-java-with-example/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/introduction-to-decorator-pattern-in-c-design-patterns/?ref=lbp> (C++)
- 5. **Facade Pattern -**
 - a. <https://www.geeksforgeeks.org/facade-design-pattern-introduction/?ref=lbp> (Java)
 - b. <https://www.geeksforgeeks.org/facade-method-c-design-patterns/?ref=lbp> (C++)
- 6. **Flyweight Pattern -**
 - a. <https://www.geeksforgeeks.org/flyweight-design-pattern/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/flyweight-pattern-c-design-patterns/?ref=lbp> (C++)
- 7. **Proxy Pattern -**
 - a. <https://www.geeksforgeeks.org/proxy-design-pattern/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/proxy-pattern-c-design-patterns/?ref=lbp> (C++)

Behavioral Design Pattern

- 1. **Chain of Responsibility Pattern -**
 - a. <https://www.geeksforgeeks.org/chain-responsibility-design-pattern/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/chain-of-responsibility-method-design-patterns-in-c/?ref=lbp> (C++)
- 2. **Command Design Pattern -**

- a. <https://www.geeksforgeeks.org/command-pattern/?ref=lbp> (Java)
 - b. <https://www.geeksforgeeks.org/command-pattern-c-design-patterns/?ref=lbp> (C++)
- 3. Interpreter Design Pattern
 - a. <https://www.geeksforgeeks.org/interpreter-design-pattern/?ref=lbp> (java)
- 4. **Observer Design Pattern**
 - a. <https://www.geeksforgeeks.org/observer-pattern-set-1-introduction/?ref=lbp> (Java)
 - b. <https://www.geeksforgeeks.org/observer-pattern-c-design-patterns/?ref=lbp> (C++)
- 5. **Strategy Design Pattern**
 - a. <https://www.geeksforgeeks.org/strategy-pattern-set-1/?ref=lbp> (Java)
 - b. <https://www.geeksforgeeks.org/strategy-method-design-pattern-c-design-patterns/?ref=lbp> (C++)
- 6. Visitor Design Pattern
 - a. <https://www.geeksforgeeks.org/visitor-design-pattern/?ref=lbp> (java)
 - b. <https://www.geeksforgeeks.org/visitor-method-design-patterns-in-c/?ref=lbp> (C++)

Low Level Design Interview Questions

- 1. Design Parking Lot -